

Entregable 1 – Monitor

Julen García Muñoz
Alberto Olcina Calabuig
Steven Silva Gómez
Daniel Banegas Reyes
Juan García Antón

14 de mayo de 2026

Índice

1. Módulo Monitor	4
1.1. Descripción General	4
1.2. Constantes	4
1.2.1. Constantes NTC	4
1.2.2. Canales ADC	4
1.3. Tipos de Datos	5
1.3.1. Estructura sensor_t	5
1.3.2. Enumeración alarm_state_t	5
1.4. Variables Globales	5
1.4.1. Sensores	5
1.4.2. Control de Sensores	5
1.4.3. LEDs	6
1.5. Funciones Privadas	6
1.5.1. ADC_ReadChannel()	6
1.5.2. Update_Sensors()	7
1.5.3. Update_Display()	7
1.5.4. Update_Alarm()	7
1.5.5. Process_Buttons()	8
1.6. Funciones Públicas	8
1.6.1. Monitor_Init()	8
1.6.2. Monitor_Loop()	9
2. Flujo de Operación	9
3. Sistema de Tareas de FreeRTOS	9
3.1. Tareas del Sistema	9
3.1.1. Task_CONSOLE	9
3.1.2. Task_TIME	10
3.1.3. Task_COMM	10
3.1.4. Task_EJER3	11

3.1.5. StartDefaultTask	11
3.2. Sincronización Entre Tareas	11
4. Flujo de Operación	12

1. Módulo Monitor

1.1. Descripción General

El módulo `monitor.c` implementa un sistema de monitoreo de sensores para una placa STM32F411 con interfaz de LEDs, buzzer y botones. El sistema permite la lectura de dos sensores (LDR para luminosidad y NTC para temperatura), ajuste de niveles de alarma mediante potenciómetro, visualización en LEDs y activación de alarma sonora.

El módulo está integrado en un sistema con **FreeRTOS** que proporciona múltiples tareas concurrentes:

- **Task_CONSOLE:** Gestiona la impresión por UART mediante un buffer thread-safe con función `bprintf()`
- **Task_TIME:** Consulta un servidor de tiempo remoto para sincronizar el reloj del sistema
- **Task_COMM:** Gestiona la comunicación WiFi mediante módulo ESP32
- **Task_EJER3:** Tarea de aplicación que imprime información del sistema
- **StartDefaultTask:** Ejecuta continuamente `Monitor_Loop()` para procesamiento de sensores

1.2. Constantes

1.2.1. Constantes NTC

```
1 #define R25      10000.0f // Resistencia a 25 deg C en ohmios
2 #define T25      298.15f  // Temperatura de referencia en Kelvin
   (25 deg C)
3 #define BETA     3900.0f  // Coeficiente Beta del termistor
```

Listing 1: Parámetros del termistor NTC

1.2.2. Canales ADC

```
1 #define CH_LDR   ADC_CHANNEL_0 // Canal para sensor LDR
2 #define CH_NTC   ADC_CHANNEL_1 // Canal para sensor NTC
3 #define CH_POT   ADC_CHANNEL_4 // Canal para potenciómetro
```

Listing 2: Asignación de canales ADC

1.3. Tipos de Datos

1.3.1. Estructura sensor_t

```
1 typedef struct {
2     float valor;           // Valor actual del sensor
3     float minimo;         // Valor mínimo de rango
4     float maximo;         // Valor máximo de rango
5     float nivel_alarma;    // Umbral de activación de
6     alarma                // alarma
7     int activated;         // Flag de activación
8     uint32_t time_activation; // Timestamp de activación
9     int value_flashing;    // Estado del parpadeo
10    uint32_t flashing_last_time; // Timestamp del último cambio
11                                de parpadeo
12 } sensor_t;
```

Listing 3: Estructura para almacenar datos de sensores

1.3.2. Enumeración alarm_state_t

```
1 typedef enum {
2     ALARM_IDLE,           // Alarma inactiva
3     ALARM_ACTIVE,         // Alarma activada/sonando
4     ALARM_COOLDOWN        // Período de enfriamiento (10 segundos)
5 } alarm_state_t;
```

Listing 4: Estados posibles de la alarma

1.4. Variables Globales

1.4.1. Sensores

```
1 sensor_t sensor_ldr = {0.0f, 0.0f, 100.0f, 50.0f, 0, 0, 0, 0};
2 // valor=0, min=0%, max=100%, umbral=50%
3
4 sensor_t sensor_ntc = {0.0f, 25.0f, 30.0f, 27.5f, 0, 0, 0, 0};
5 // valor=0, min=25 deg C, max=30 deg C, umbral=27.5 deg C
```

Listing 5: Instancias globales de sensores

1.4.2. Control de Sensores

```
1 uint8_t selected_sensor = 0;           // Sensor activo: 0=LDR, 1=
2     NTC
3
4 uint32_t last_pot_value = 0xFFFF;      // Valor anterior del potenci
5     ómetro
```

```

3 alarm_state_t alarm_state = ALARM_IDLE;      // Estado de la
   alarma
4 uint32_t alarm_cooldown_start = 0;  // Timestamp del inicio del
   cooldown
5 uint8_t btn_izq_last = 1;            // Estado anterior botón
   izquierdo
6 uint8_t btn_der_last = 1;           // Estado anterior botón
   derecho
7 uint32_t bajadaIZQ;                 // Timestamp cuando se
   presiona botón izq
8 uint32_t bajadaDER;                 // Timestamp cuando se
   presiona botón der
9 uint32_t subidaIZQ;                 // Timestamp cuando se suelta
   botón izq
10 uint32_t subidaDER;                 // Timestamp cuando se suelta
   botón der
11 uint8_t g_mode;                     // Modo del sistema: 0, 1 o 2

```

Listing 6: Variables de control

1.4.3. LEDs

```

1 GPIO_TypeDef* LED_PORT[8] = {LED8_GPIO_Port, LED7_GPIO_Port,
2                               LED6_GPIO_Port, LED5_GPIO_Port,
3                               LED4_GPIO_Port, LED3_GPIO_Port,
4                               LED2_GPIO_Port, LED1_GPIO_Port};
5 uint16_t LED_PIN[8] = {LED8_Pin, LED7_Pin, LED6_Pin, LED5_Pin,
6                        LED4_Pin, LED3_Pin, LED2_Pin, LED1_Pin};

```

Listing 7: Mapeo de pines LED

1.5. Funciones Privadas

1.5.1. ADC_ReadChannel()

```

1 static uint32_t ADC_ReadChannel(uint32_t channel)

```

Listing 8: Lectura de canal ADC

Descripción: Lee el valor digital de un canal ADC específico.

Parámetros:

- **channel:** identificador del canal ADC a leer

Retorna: Valor digital de 12 bits (0-4095)

Detalles:

- Configura el canal

- Inicia conversión ADC
- Espera a que la conversión termine
- Obtiene el valor y detiene el ADC

1.5.2. Update_Sensors()

```
1 static void Update_Sensors(void)
```

Listing 9: Actualización de valores de sensores

Descripción: Actualiza los valores de los sensores LDR, NTC y el potenciómetro.

Funcionalidad:

- **LDR:** Convierte lectura ADC a porcentaje (0-100 %), invertida (mayor ADC = menor luminosidad)
- **NTC:** Calcula temperatura en °C usando ecuación de Steinhart-Hart
- **Potenciómetro:** Ajusta el nivel de alarma del sensor actualmente seleccionado con histéresis de 30 unidades para evitar ruido

1.5.3. Update_Display()

```
1 static void Update_Display(void)
```

Listing 10: Actualización de visualización en LEDs

Descripción: Controla el encendido/apagado de LEDs según el valor del sensor activo.

Funcionalidad:

- Calcula cuántos LEDs deben estar encendidos basándose en el valor relativo del sensor
- Identifica el LED que corresponde al umbral de alarma
- Genera parpadeo a 10 Hz en el LED del umbral
- Actualiza el estado de todos los LEDs

1.5.4. Update_Alarm()

```
1 static void Update_Alarm(void)
```

Listing 11: Actualización del estado de alarma

Descripción: Gestiona la máquina de estados de la alarma.

Estados:

- **ALARM_IDLE:** Monitorea si algún sensor supera su umbral
- **ALARM_ACTIVE:** Alarma sonando, se mantiene hasta pulsar botón derecho
- **ALARM_COOLDOWN:** Período de 10 segundos antes de poder activar nuevamente

1.5.5. Process_Buttons()

```
1 static void Process_Buttons(void)
```

Listing 12: Procesamiento de entrada de botones

Descripción: Detecta pulsaciones registrando timestamps en bajada/subida de botones y discrimina por duración.

Lógica:

- Registra **bajada*** al flanco descendente (presionar) y **subida*** al ascendente (soltar)
- Si duración < 2 s: pulsación corta
- Si duración $\geq$ 2 s: pulsación larga

Acciones:

- **Botón IZQ corta:** Alterna sensor (LDR $\leftrightarrow$ NTC)
- **Botón DER corta:** Silencia alarma, entra en cooldown
- **Ambos $\geq$ 2s:** Avanza modo del sistema (`g_mode++`)

1.6. Funciones Públicas

1.6.1. Monitor_Init()

```
1 void Monitor_Init(void)
```

Listing 13: Inicialización del módulo

Descripción: Inicializa el módulo de monitoreo.

Funcionalidad:

- Apaga todos los LEDs (GPIO_PIN_RESET)
- Apaga el buzzer

Debe llamarse: Una vez en la función `main()` durante la inicialización del sistema.

1.6.2. Monitor_Loop()

```
1 void Monitor_Loop(void)
```

Listing 14: Bucle principal de monitoreo

Descripción: Función principal que debe llamarse periódicamente desde la tarea StartDefaultTask.

Contexto de Ejecución: Se ejecuta en una **tarea de FreeRTOS** denominada StartDefaultTask con prioridad NORMAL.

Ciclo de ejecución: Se ejecuta cada 20 ms (50 Hz) mediante temporizador interno.

Secuencia de operaciones:

1. Process_Buttons(): Procesa entrada de botones
2. Update_Sensors(): Actualiza valores de sensores
3. Update_Alarm(): Gestiona estado de alarma
4. Update_Display(): Actualiza visualización en LEDs

Temporizador: Utiliza HAL_GetTick() para asegurar una ejecución consistente cada 20 ms.

2. Flujo de Operación

3. Sistema de Tareas de FreeRTOS

El proyecto utiliza **FreeRTOS** para gestionar múltiples tareas concurrentes. La inicialización se realiza a través de la función CONFIGURACION_INICIAL() llamada desde main.c.

3.1. Tareas del Sistema

3.1.1. Task_CONSOLE

- **Prioridad:** NORMAL_PRIORITY
- **Stack:** 2048 bytes
- **Función:** Gestiona la salida de datos por UART2 mediante un buffer thread-safe
- **Interfaz de Impresión:** Proporciona la función bprintf() para impresión formateada desde cualquier tarea

- **Características:**

- Buffer circular con capacidad de 128 caracteres
- Función `bprintf_last()` para mensajes prioritarios
- Funciones de sincronización: `console_lock()`, `console_unlock()`

3.1.2. Task_TIME

- **Prioridad:** NORMAL_PRIORITY
- **Stack:** 2048 bytes
- **Función:** Solicita y sincroniza la hora del sistema desde un servidor remoto
- **Servidor:** pperez2.disca.upv.es:5000
- **Protocolo:** HTTP GET /tiempo
- **Procesamiento:**
 - Parsea respuesta JSON con cJSON
 - Busca campo "tiempo_actual" con formato: AAAA-MM-DD HH:MM:SS
 - Convierte a `time_t` usando `mktime()`
 - Mantiene sincronización relativa usando `xTaskGetTickCount()`
- **Interfaz:**
 - `task_TIME_timeAvailable()`: Indica si la hora es válida
 - `task_TIME_getTime()`: Retorna la hora actual en `time_t`
- **Período:** Solicita actualización cada 100 segundos
- **Sincronización:** Usa semáforo mutex `COMM_xSem` para acceso exclusivo

3.1.3. Task_COMM

- **Prioridad:** NORMAL_PRIORITY
- **Stack:** 2048 bytes
- **Función:** Gestiona la comunicación WiFi mediante módulo ESP32
- **Configuración WiFi:**
 - SSID: routerSEU
 - Contraseña: 00000000
- **Interfaz de Solicitudes:**
 - Estructura `COMM_request` para pasar solicitudes desde otras tareas

- Campo **command**: 0=inactivo, 1=solicitud pendiente, 2=procesando
- Campo **result**: 1 cuando la respuesta está lista
- **Procesamiento:**
 - Envía solicitud HTTP mediante `ESP_Send_Request()`
 - Limpia encabezados HTTP de la respuesta
 - Preserva solo el contenido JSON puro
- **Sincronización:** Usa semáforo mutex con timeout de 20 segundos

3.1.4. Task_EJER3

- **Prioridad:** NORMAL_PRIORITY
- **Stack:** 2048 bytes
- **Función:** Tarea de aplicación que imprime información del sistema
- **Período:** 10 segundos
- **Salida:**
 - Si hora disponible: [HH:MM:SS] Cnt=X
 - Si hora no disponible: Hora no disponible. Cnt=X
 - Estado del modo: `g_mode`: X

3.1.5. StartDefaultTask

- **Prioridad:** NORMAL_PRIORITY
- **Stack:** 512 bytes
- **Función:** Ejecuta continuamente `Monitor_Loop()`
- **Período:** Ajustable mediante `osDelay(1)` (típicamente 1-2 ms entre ciclos)

3.2. Sincronización Entre Tareas

El sistema utiliza varios mecanismos de sincronización:

- **Mutex COMM_xSem:** Protege acceso a la estructura `COMM_request`
- **Variables Globales:**
 - `g_mode`: Modo del sistema, modificable por pulsación de botones
 - `time_available`: Indica disponibilidad de hora
 - `global_wifi_ready`: Estado de conexión WiFi
- **Buffer Thread-Safe IObuff:** Maneja impresión desde múltiples tareas sin conflictos

4. Flujo de Operación

El sistema opera de la siguiente manera:

1. Inicialización:

- Se llama a `Monitor_Init()` para apagar todos los LEDs y buzzer
- Se llama a `CONFIGURACION_INICIAL()` para crear todas las tareas de FreeRTOS
- Se inicia el planificador de FreeRTOS

2. Lectura de Sensores (`StartDefaultTask`):

El módulo monitor lee continuamente los tres canales ADC en cada ciclo de 20 ms (LDR, NTC, Potenciómetro)

3. Procesamiento de Sensores:

Los valores se convierten a unidades significativas (porcentaje, °C) y se normaliza el rango de alarma mediante el potenciómetro

4. Control de Alarma:

Se comparan los valores con los umbrales ajustables:

- Si sensor supera umbral: Alarma activa, buzzer suena
- Si se pulsa botón derecho: Alarma silenciada, entra en cooldown de 10 segundos
- Después de cooldown: Sistema vuelve a estado IDLE

5. Visualización (LEDs):

Los 8 LEDs muestran el nivel relativo del sensor activo, con parpadeo (10 Hz) en el LED que marca el umbral de alarma

6. Interacción de Botones:

- Botón izquierdo (corta pulsación): Cambia entre sensor LDR y NTC
- Botón derecho (corta pulsación): Silencia alarma si está activa
- Ambos botones (larga pulsación ≥ 2 s): Cambia modo del sistema (`g_mode`)

7. Sincronización de Hora (`Task_TIME`):

- Cada 100 segundos solicita hora al servidor
- Parsea respuesta JSON
- Actualiza variable global `time_available`

8. Impresión de Información (`Task_EJER3`):

- Cada 10 segundos imprime estado usando `bprintf()`
- Muestra hora sincronizada si disponible
- Imprime modo actual del sistema